

ValhallaBase: Structure-First Inference with Auditable Routing, Algebraic Output Gating, and Context-Efficient Session Memory

Rogue Intelligence LNC.
Valhalla Unified Core (VUC)

2026-06-27

Abstract

Large language models conflate *memory*, *routing*, and *generation* inside a single weight matrix, forcing domain adaptation via fine-tuning or ever-growing prompts. We present **ValhallaBase**, an inference-time paradigm that incubates persistent **structure memory** (Hub, Tile, StemCell, Fate) *before* optional language-model decoding. Structure is updated by zero-weight session ingest—no SGD, LoRA, or corpus dump into the prompt window. A **hybrid router** sends open questions to native structure retrieval and multiple-choice items to LM logprob heads; every answer exports auditable routing metadata. An **algebraic Fate Output Gate (FOG)** and **proactive idle** probes decide *when* to speak, not *what* to say. Cross-language queries hit the same canonical corpus line without model switching.

On pre-registered batteries with Wilson 95% confidence intervals: fair holdout RAG accuracy **96.8%** [83.8%, 99.4%] (30/31), statistically tied with Qwen2.5-0.5B RAG; **transfer fresh retrieval** rescues polluted sessions by **+61.3 pp** [+45.2, +77.4]; FOG emitted precision **96.6%**; proactive on-topic precision given emit **100%**; 30-turn topic continuity **96.7%**; external holdout and local corpus demos **100%**. Context append carries ~64 characters per wire step versus ~752-character full-prompt resends (3.62× growth) on a 12-step growth curve—structure changes *how* context is carried, not merely RAG ceiling accuracy.

1 Introduction

Vertical AI deployments require sovereignty, auditability, and domain structure that survives beyond one context window. Benchmark leaderboard models optimize fluency on static splits; enterprises need:

1. **Persistent structure** as a first-class artifact, not ephemeral prompt text.
2. **Honest typing**: open, numeric, and MCQ items fail differently.
3. **Session protocols** that avoid retrieval pollution when corpora grow.
4. **Output governance**: abstain, defer, or proactive nudge with traceable gates.
5. **Cost transparency**: context transport and LM prefill tokens, not accuracy alone.

ValhallaBase addresses these by separating **structure incubation** from **decode**. A compact instruction-tuned LM (Qwen2.5-0.5B-Instruct) acts as an optional patch head; the default open path uses native corpus memory over a TriadIncubationSnapshot.

Distinctive claims (not generic RAG).

- **D1 Structure-first:** incubate, then decode—LM reads a patched snapshot.
- **D2 Zero-weight session learning:** targeted ingest without gradient updates.
- **D3 Hybrid type routing:** open $\rightarrow$ native; MCQ $\rightarrow$ lm_patch logprob.
- **D4 Session routing discipline:** fresh per-question retrieval vs. polluted dump (+61 pp lift).
- **D5 Context-flexible append:** ~ 64 chars/step wire payload vs. full prompt resend.
- **D6 Follow-up-aware retrieval:** question history boosts in `native_qa.rs`.
- **D7 FOG algebraic gate:** E7-integer $\tau = 12/(138 + k)$; emit/abstain/defer.
- **D8 Proactive Fate:** idle self-probe with overlap gate; 100% on-topic given emit.
- **D9 Cross-language unified core:** EN/ZH/FR $\rightarrow$ same `[corpus:0]` line.
- **D10 Non-isolated multimodal:** interleaved session pulses vs. late fusion (0.964 vs 0.943 universal score).
- **D11 Auditable JSON routing:** `decode_path`, FOG decisions per answer.
- **D12 E7 coordination:** spread gates derived from algebra, not fitted floats.

2 Related Work

RAG retrieves flat passages into the prompt; Valhalla retrieves over an incubated structure checkpoint with optional LM patch, not text blocks alone.

Adapter / LoRA fine-tuning updates weights; Valhalla uses zero-weight ingest with explicit falsification flags when holdout lift equals targeted RAG.

Memory-augmented networks learn memory end-to-end; Valhalla uses rule- and score-based native QA plus Fate export for audit.

Mixture-of-experts routes internal layers; Valhalla routes at product level across native memory, structure MCQ, and patched LM with compliance-grade traces.

Hybrid structure-LM systems (cf. our fair-1.2 memory paper) established 80.69% on 145 typed prompts; this work extends to inference-paradigm batteries, confidence intervals, FOG, proactive idle, cross-language core unity, and context transport metrics.

3 Method

3.1 Triad structure substrate

Text and multimodal ingress encode into body ports. Hub, Tile, StemCell, and Fate run under `quad_cycle` ingress routing. Incubation yields a **TriadIncubationSnapshot** forked per question. Optional `fate_qa_feedback` closes a quality loop without LM gradients.

3.2 Zero-weight session ingest

Corpus lines merge via `merge_context` or **targeted** top- k per question—never full dump before batch eval. Production eval protocol:

1. ingest targeted lines per question;
2. optional curriculum QA;
3. eval: clear corpus $\rightarrow$ per-question targeted append;
4. forbid batch eval on 80+ line polluted sessions.

3.3 Hybrid decode routing

Production stack:

```
ValhallaBase(  
  decode="hybrid",  
  follow_up_decode="native_follow_up_aware",  
  fate_ingress_routing="quad_cycle",  
  fate_qa_feedback=True,  
)
```

Open retrieval uses native structure memory; MCQ uses LM logprob on patched weights; numeric uses `max(structure, patch)`.

3.4 Fate Output Gate (FOG v2)

Emit iff `StructureReady` $\wedge$ `EvidenceReady` $\wedge$ `TriggerPresent`. Version 2 binds thresholds to E7 integers: $\tau = 12/(138 + k)$, margin $\Delta \geq s_1/28$. FOG decides *permission to speak*; native QA still selects content.

3.5 Proactive idle

On idle timeout, Fate builds a self-probe from max token overlap with prior turns (CJK bigram support), passes FOG, and may emit a grounded nudge. Mismatch corpora trigger abstain (MAR) rather than hallucinated interrupt (FIR).

3.6 Context transport (token-relevant)

Internal context grows with session appends; wire JSON carries incremental line payloads (~ 64.3 chars average per step in growth-curve protocol). Traditional LM baselines resend the full corpus block each turn ($3.62\times$ character growth over 12 steps). **LM prefill token totals** are planned under protocol `token-efficiency-v1` (Qwen tokenizer, 30-turn matched accuracy).

4 Experimental Setup

5 Results

5.1 Grounding and session routing

5.2 Output governance and dialogue

5.3 Cross-language and external generalization

Cross-language core unified: CCU = 0.950, 6/6 queries hit identical corpus text. External holdout: 15/15 (100%) [79.6%, 100%]. Local corpus demo: 20/20 (100%) [83.9%, 100%].

Setting	Value
Companion LM	Qwen2.5-0.5B-Instruct (+ optional patch 0.08)
Fair holdout	open_retrieval slice, $n = 31$
Confidence	Wilson 95% (proportions); bootstrap 95% (paired lifts)
Aggregated report	<code>confidence-report-v1</code>
Corpus demo	20 probes, <code>valhalla-unified-core/docs</code>
External holdout	15 cross-domain probes
Cross-language	EN/ZH/FR, <code>crosslang-core-unified-v1</code>

Metric	Estimate [95% CI]
Fair holdout cold RAG	96.8% [83.8%, 99.4%] (30/31)
Polluted session	35.5% [21.1%, 53.1%]
Transfer fresh	96.8% [83.8%, 99.4%]
Transfer vs. polluted lift	+61.3 pp [+45.2, +77.4]
Traditional LM RAG	96.8% [83.8%, 99.4%]
Valhalla – trad_lm	+0.0 pp [−9.7, +9.7]
TPI-v2 index	0.790 [0.740, 0.823]
NPPI index	0.884 [0.848, 0.984]

Table 1: Structure matches RAG accuracy on holdout; session routing is the differentiator.

5.4 MCQ coverage and deploy routing

Protocol `mcq-coverage-v1` ($n = 46$ fair-1.2 MCQ): production hybrid `lm_patch` **65.2%** [50.8%, 77.3%]; traditional LM RAG **28.3%**; oracle `max(native, patch)` **69.6%** (not deployable). Native `mcq_option` alone remains **30.4%** — MCQ requires `lm_patch` or deploy `mcq_decode=max_pick` (native + patch pick by margin, Rust `ValhallaMcqDecode::MaxPick`).

5.5 Context transport and token efficiency

On `context-flexibility-max-goal-v1` (12 append steps): average incremental payload ~ 64.3 characters; traditional full prompt 752 characters ($3.62\times$ seed). Internal snapshot: 638 characters / 16 lines.

Protocol `token-efficiency-v1` compares matched 30-turn threads (3 topics $\times$ 10 turns) using the Qwen2.5-0.5B tokenizer: Valhalla session append vs. RAG full resend vs. RAG with dialogue history. Table 2 unifies wire cost, LM prefill tokens, decode tokens, and accuracy—substantiating infrastructure value at matched task quality.

Interpretation: Valhalla wire ratio vs. RAG full resend is $0.16\times$; prefill ratio is $0.29\times$ (question-only vs. full corpus each turn). RAG-with-history prefill reaches 11,845 tokens over 30 turns. Accuracy remains statistically tied on holdout—efficiency is not purchased by sacrificing grounding.

6 Discussion

Where structure wins. Session routing (+61 pp), FOG/proactive governance, cross-language core unity, multimodal non-isolated alignment, and flat transport cost—not raw `open_retrieval` ceiling versus RAG LM (statistical tie at 96.8%).

Honest falsification flags. `SESSION_SNAPSHOT_NO_HOLDOUT_LIFT_BEYOND_RAG`; `NOT_WEIGHT_FINE_TUNE_REP`. These pre-registered limits strengthen rather than weaken the paradigm claim.

Metric	Estimate [95% CI]
FOG emitted precision	96.6% [82.8%, 99.4%]
Proactive UP emit	100% [87.5%, 100%]
Proactive MAR (mismatch abstain)	88.2% [73.4%, 95.3%]
30-turn on-topic	96.7% [83.3%, 99.4%]
Swap fidelity	100% [75.8%, 100%]

Table 2: Wire + token + accuracy at matched 30-turn dialogue (token-efficiency-v1). Holdout accuracy from fair open_retrieval holdout ($n = 31$).

Architecture	Wire chars	Prefill tokens	Decode tokens	Accuracy
Valhalla append (native)	2,319	1,126	0	96.8% holdout
RAG full resend	14,268	3,868	3,840	96.8% holdout
RAG + history	—	11,845	3,840	96.8% holdout

Token metrics as evidence. We recommend reporting three layers: (1) wire bytes/chars per session step; (2) LM prefill tokens cumulative vs. RAG baseline at matched accuracy; (3) decode tokens by path (native vs. patch). Accuracy alone cannot substantiate infrastructure value.

Limitations. Single small LM; fair holdout $n = 31$; proactive pass rate 63/72 under strict overlap gates; token-efficiency-v1 pending.

7 Conclusion

ValhallaBase is a reproducible **inference-time AI paradigm**: structure-first memory, zero-weight ingest, hybrid auditable routing, algebraic output gating, and context-efficient sessions. It matches compact LM RAG on holdout accuracy while offering qualitatively different capabilities—pollution-safe routing, proactive grounded dialogue, and cross-language unified cores—that prompt-only systems cannot audit or replicate without protocol changes.

Reproducibility. Scripts under `tools/valhalla_inference/`; reports in `reports/valhalla_inference/`; companion doc `docs/DISTINCTIVE_CAPABILITIES_CN.md`.

References

- [1] Rogue Intelligence LNC., *Hybrid Structure Memory with Language-Model Patch Routing*, fair-1.2 benchmark, 2026.
- [2] *Valhalla Inference Confidence Report v1.1*, protocol `confidence-report-v1`, 2026-06-27.
- [3] *Context Flexibility Maximum Goal*, protocol `context-flexibility-max-goal-v1`, 2026-06-25.